Project Title

E-Voting

By

SHAMAYA WALTER

Roll No: RBSIT-21-05

KINZA ZOBI

Roll No: RBSIT-21-06

Session: 2021-2025

Supervisor: Mr. Ghulam Ghos

Department of Information Technology, Bahauddin Zakariya University Multan

January 2026

Certificate of Approval

This is to certify that the project titled "**E-Voting**" submitted by **SHAMAYA WALTER (Roll No: RBSIT-21-05)** & **KINZA ZOBI (Roll No: RBSIT-21-06)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology has been examined and approved.

Name: Mr. Ghulam Ghos

Signature: _____

Date: _____

Name: Dr. Maroof Pasha

Signature: _____

Date: _____

Declaration

I hereby declare that this project titled "**E-Voting**" is my own work and has not been submitted elsewhere for any other degree or qualification

Name: SHAMAYA WALTER

Roll No: RBSIT-21-05

Signature: _____

Date: _____

Name: KINZA ZOBI

Roll No: RBSIT-21-06

Signature: _____

Date: _____

Acknowledgements

All things come from You, and of Your own have we given You." — **1 Chronicles 29:14**
All praise and glory be to **God Almighty**, for by His abundant grace and mercy, I have been able to accomplish this work. I thank the Lord for His guidance, strength, and countless blessings that sustained me throughout this journey.

I am deeply grateful to **Jesus Christ**, our Lord and Savior, for being my constant source of wisdom, hope, and courage. As it is written, "I can do all things through Christ who strengthens me" (**Philippians 4:13**), this truth carried me through every challenge during my studies.

My heartfelt thanks go to my father, whose encouragement and steadfast support shaped my determination and perseverance. Truly, "a wise son brings joy to his father" (**Proverbs 10:1**). I also extend special gratitude to my mother, whose prayers, love, and guidance strengthened my confidence and kept me focused on my goals. Her care for my health, education, and future has been a blessing beyond measure.

I am grateful to my siblings, whose support, advice, and encouragement were invaluable. Especially, I thank my brothers, who guided me and stood by me throughout this journey.

I would like to express my sincere appreciation to my supervisor, **Mr. Ghulam Ghos**, for his patience, knowledge, and continuous guidance. His mentorship was a true reflection of Proverbs 15:22: "Plans fail for lack of counsel, but with many advisers they succeed."

I also sincerely thank **Mam Aliya Bodla**, Principal and Director of the Regional Institute of Allied Health Sciences and Higher Education, Mian Channu, for her concern, support, and dedication to students' success.

Finally, I acknowledge the assistance and encouragement of **Mr. Usam,** whose help was a source of motivation and strength throughout this work.

To God be the glory, for all things have been made possible by Him. "Now to Him who is able to do immeasurably more than all we ask or imagine, according to His power that is at work within us" (**Ephesians 3:20**).

SHAMAYA WALTER
RBSIT-21-05

KINZA ZOBI
RBSIT-21-06

Abstract

This project presents a secure and efficient **Fingerprint-Based E-Voting System** designed to improve the transparency, accuracy, and reliability of the voting process. Traditional voting methods often face challenges such as impersonation, duplicate voting, lengthy vote counting, and human errors. To overcome these issues, the proposed system integrates biometric fingerprint authentication with a web-based voting platform.

The system is developed using **HTML, CSS, JavaScript, Python (Flask), and MongoDB**. It allows voters to register using their personal information and fingerprint, authenticate themselves securely, and cast only one vote during the scheduled election period. The administrator can manage voters, candidates, and election schedules while monitoring live election results through an interactive dashboard. The system also provides server-side validation, secure session management, and protection against unauthorized access and duplicate voting.

The proposed solution demonstrates that biometric authentication combined with modern web technologies can provide a reliable and user-friendly electronic voting platform for universities, organizations, and other institutions. The project successfully achieves its objectives by ensuring secure voter identification, accurate vote recording, and real-time result visualization while reducing the limitations of traditional paper-based voting systems.

Keywords: E-Voting System, Fingerprint Authentication, Biometric Security, Flask, MongoDB, Web Application, Electronic Voting.

TABLE OF CONTENTS

List of FIGURES

Chapter 1: Introduction

1.1 Project Overview

The E-Voting System is a fingerprint-based secure electronic voting platform developed to modernize and streamline the electoral process. The system combines biometric authentication, a time-scheduled voting window, real-time result visualization, and a comprehensive administrative panel into a single cohesive web application. Built with Python (Flask) on the backend and plain HTML/CSS/JavaScript on the frontend, it uses MongoDB as its database engine and integrates with the SecuGen HU20 fingerprint scanner hardware through the SecuGen BWAPI (Browser Web API).

The platform supports the complete lifecycle of a digital election: voter registration with fingerprint enrollment, identity-verified login, ballot casting within a scheduled window, live results display, and admin-controlled management of candidates, voters, and schedules. Every sensitive operation is protected by server-side validation, ensuring that no client-side manipulation can bypass the security constraints.

1.2 Problem Statement

Traditional paper-based voting systems suffer from numerous well-documented problems: ballot stuffing, impersonation of voters, double voting, slow manual counting, and a lack of real-time transparency. In the Pakistani context, these challenges are particularly acute given geographic spread, logistical difficulties, and the administrative overhead of managing large voter rolls.

Existing digital voting solutions often rely on PIN or password authentication alone, which is susceptible to sharing, theft, and phishing. They also tend to lack integrated schedule enforcement — allowing votes to be cast outside the official window — and provide no real-time audit trail for administrators. There is a clear need for a system that combines biometric identity verification, tamper-resistant vote recording, and transparent live result reporting.

1.3 Project Motivation

The motivation for this project arises from three converging needs. First, the democratization of technology access means that a web-based voting solution is now a realistic proposition for institutional elections at universities, local bodies, or professional organizations. Second, the falling cost of USB fingerprint scanners (such as the SecuGen HU20) makes biometric authentication affordable at scale. Third, the availability of mature open-source frameworks (Flask, MongoDB, Chart.js) enables a small team to deliver a production-quality system without enterprise infrastructure.

Beyond the technical opportunity, there is an ethical imperative: every valid vote must be counted exactly once, and every fraudulent vote must be rejected. This project treats that requirement as a hard constraint, not a best-effort goal, and designs every layer of the system to enforce it.

1.4 Aims and Objectives

The primary aim of this project is to design and implement a fully functional, secure, and user-friendly electronic voting system that replaces paper ballots with a biometrically authenticated digital workflow.

Specific objectives include:

- Implement a voter registration module that captures personal details and enrolls a fingerprint template using the SecuGen HU20 scanner.
- Develop a login mechanism that authenticates voters using CNIC number combined with a live fingerprint scan, eliminating the need for remembered passwords.
- Enforce a time-locked voting schedule so that votes can only be cast within an admin-defined window, preventing early or late submissions.
- Guarantee single-vote integrity through server-side has Voted flag validation, making double voting impossible regardless of client-side behavior.
- Record GPS coordinates at the time of voting and reverse-geocode them to city/area level using the OpenStreetMap Nominatim API.
- Provide a live results dashboard with bar charts, candidate rankings, and voter turnout statistics that update every five seconds.
- Build a comprehensive admin panel allowing management of candidates, voters, and schedules, with CSV export and vote reset capabilities.
- Ensure graceful degradation when the BWAPI fingerprint service is unavailable by falling back to a byte-similarity comparison algorithm suitable for development and testing.

1.5 Project Scope

The system is scoped for institutional elections with a single election active at any given time. It supports an unlimited number of registered voters and candidates. The voter interface is web-based and requires a modern browser; no native mobile application is included. The admin panel is password-protected and accessible from the same web server.

The system operates entirely on a local network (localhost) in its default configuration. Deployment to a public-facing server requires additional hardening (HTTPS, reverse proxy, environment variable management) that is documented in the appendix but is outside the primary scope.

Out of scope for this version: postal / remote voting without physical scanner access, multi-election concurrency, and voter anonymization beyond what MongoDB document-level access control provides.

1.6 Document Structure

The remainder of this report is organized as follows:

- Chapter 2 provides a literature review of existing e-voting systems and identifies gaps addressed by this project.
- Chapter 3 presents requirement analysis, stakeholder identification, functional and non-functional requirements, use cases, and user stories.
- Chapter 4 describes the system design including architecture, database schema, API design, process flows, and security design.
- Chapter 5 details the implementation, covering the development environment, backend modules, frontend modules, and key code explanations.
- Chapter 6 presents the testing strategy and results, including functional test cases and edge case testing.
- Chapter 7 discusses the results, performance evaluation, and limitations.
- Chapter 8 concludes the report and outlines future enhancement opportunities.

Chapter 2: Literature Review & Background Study

2.1 Introduction to the Domain

Electronic voting (e-voting) encompasses any election technology that uses electronic means to cast or count votes. The spectrum ranges from electronic counting of paper ballots, through direct-recording electronic (DRE) terminals, to fully internet-based remote voting. Each point on the spectrum trades off accessibility against security and verifiability.

Biometric authentication, particularly fingerprint recognition, has emerged as a strong authentication mechanism for voting systems because fingerprints are inherently bound to the individual, cannot be forgotten or shared in the same way as passwords, and can be verified in under a second with modern SDK implementations. ISO/IEC 19794-2 defines the standard for fingerprint minutiae data, and the SecuGen SDK used in this project complies with that standard.

2.2 Existing Systems Analysis

Several e-voting systems have been studied to inform the design of this project:

National Electronic Voting System (EVS) — Pakistan: The Election Commission of Pakistan has piloted tablet-based polling stations with barcode voter verification. While operationally effective, these systems rely on network connectivity to a central server and use barcode scanning rather than biometrics, leaving them vulnerable to card theft impersonation.

Estonia's I-Voting System: Estonia operates a nationally deployed internet voting system where voters authenticate using a national ID smart card or Mobile-ID. While highly sophisticated, it requires a PKI (Public Key Infrastructure) that is impractical for institutional elections. Voters can change their vote during the voting window, a feature that introduces complexity absent in this project.

Open-Source EVS (OSCAR, Helios): Academic and open-source systems such as Helios focus on cryptographic verifiability (end-to-end verifiable elections). They typically do not include biometric authentication and require significant cryptographic expertise to deploy and audit.

2.3 Comparative Analysis

System	Auth Method	Biometric	Schedule Enforcement	Real-Time Results	Open Source

System	Auth Method	Biometric	Schedule Enforcement	Real-Time Results	Open Source
Pakistan EVS Pilot	Barcode Card	No	Manual	No	No
Estonia I-Voting	Smart Card / PKI	No	Yes (server)	No	Partial
Helios	Password / OAuth	No	Yes	Post-election	Yes
This Project	CNIC + Fingerprint	Yes (ISO Template)	Yes (server-locked)	Yes (5s polling)	Yes

Table 2.1 – Comparison of Existing E-Voting Systems

2.4 Identified Research Gaps

From the comparative analysis, three key gaps are identified:

1. No existing accessible (low-cost, open-source) system combines biometric fingerprint authentication with a server-enforced voting schedule and live result visualization in a single deployable package.
2. Systems that do use biometrics (national-level deployments) are not designed for institutional reuse — they require dedicated infrastructure, trained operators, and procurement channels inaccessible to universities or NGOs.
3. Academic systems that prioritize cryptographic verifiability (Helios) sacrifice usability, making them impractical for general deployment without specialist expertise.

2.5 Proposed Solution Overview

This project addresses the identified gaps by building a pragmatic system that prioritizes the following properties: deploy ability (runs on any laptop with Python and MongoDB), security (fingerprint + schedule enforcement), usability (single-page voter flow in three steps), and transparency (live public results chart). The system deliberately avoids complex cryptographic constructs in favor of well-understood server-side validations, making it auditable and maintainable by developers with standard web engineering skills.

Chapter 3: Requirement Analysis & Specification

3.1 Stakeholder Analysis

Three primary stakeholder groups interact with the system:

- Voter: Any registered individual eligible to cast one vote. Primary concern is ease of use and confidence that their vote is recorded correctly.
- Election Administrator (Admin): Responsible for setting up the election — adding candidates, configuring the schedule, monitoring turnout, and resetting the system for a new election. Primary concern is control and auditability.
- Observer / Public: Anyone who wishes to view live or final results. No authentication required; results page is publicly accessible when voting is active or complete.

3.2 Functional Requirements

ID	Module	Requirement	Priority
FR-01	Registration	System shall allow a voter to register with name, age, gender, CNIC, email, password, address, fingerprint template, and optional photo.	High
FR-02	Registration	System shall reject duplicate CNIC or email registrations with a descriptive error.	High
FR-03	Registration	System shall validate CNIC format (12345-1234567-1) server-side.	High
FR-04	Registration	System shall store fingerprint template in ISO format encoded as Base64.	High
FR-05	Authentication	System shall authenticate voters using CNIC + live fingerprint scan only.	High

ID	Module	Requirement	Priority
FR-06	Authentication	System shall create a server-side session on successful login.	High
FR-07	Voting	System shall only accept votes within the admin-defined schedule window.	High
FR-08	Voting	System shall prevent a voter who has already voted from casting another vote.	High
FR-09	Voting	System shall verify fingerprint before recording a vote.	High
FR-10	Voting	System shall optionally record GPS coordinates and reverse-geocoded location.	Medium
FR-11	Results	System shall display live vote counts updated every 5 seconds.	High
FR-12	Results	System shall hide vote counts when voting has not yet started.	Medium
FR-13	Admin	System shall allow admin to add and delete candidates.	High
FR-14	Admin	System shall prevent candidate deletion while voting is active.	High
FR-15	Admin	System shall allow admin to set and update the voting schedule.	High
FR-16	Admin	System shall provide voter list with search, filter, and CSV export.	Medium
FR-17	Admin	System shall allow admin to reset all votes with a confirmation word.	High

ID	Module	Requirement	Priority
FR-18	Fingerprint	System shall fall back to byte-similarity matching when BWAPI is unavailable.	Medium

Table 3.1 – Functional Requirements

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords stored as bcrypt hashes with random salt; plaintext never persisted.
NFR-02	Security	Fingerprint templates never returned to client; stored server-side only.
NFR-03	Security	Admin and voter sessions are isolated; admin routes protected by @admin_required decorator.
NFR-04	Performance	Vote status API must respond within 500ms under normal load.
NFR-05	Usability	Voter journey from login to vote confirmation must require no more than 3 user actions.
NFR-06	Reliability	System must operate without BWAPI hardware in development/fallback mode.
NFR-07	Maintainability	Each backend module (auth, vote, result, admin, fp) must be in a separate Blueprint file.
NFR-08	Compatibility	Frontend must function on Chrome 100+ and Firefox 100+ without build tools.

Table 3.2 – Non-Functional Requirements

3.4 Use Case Modeling

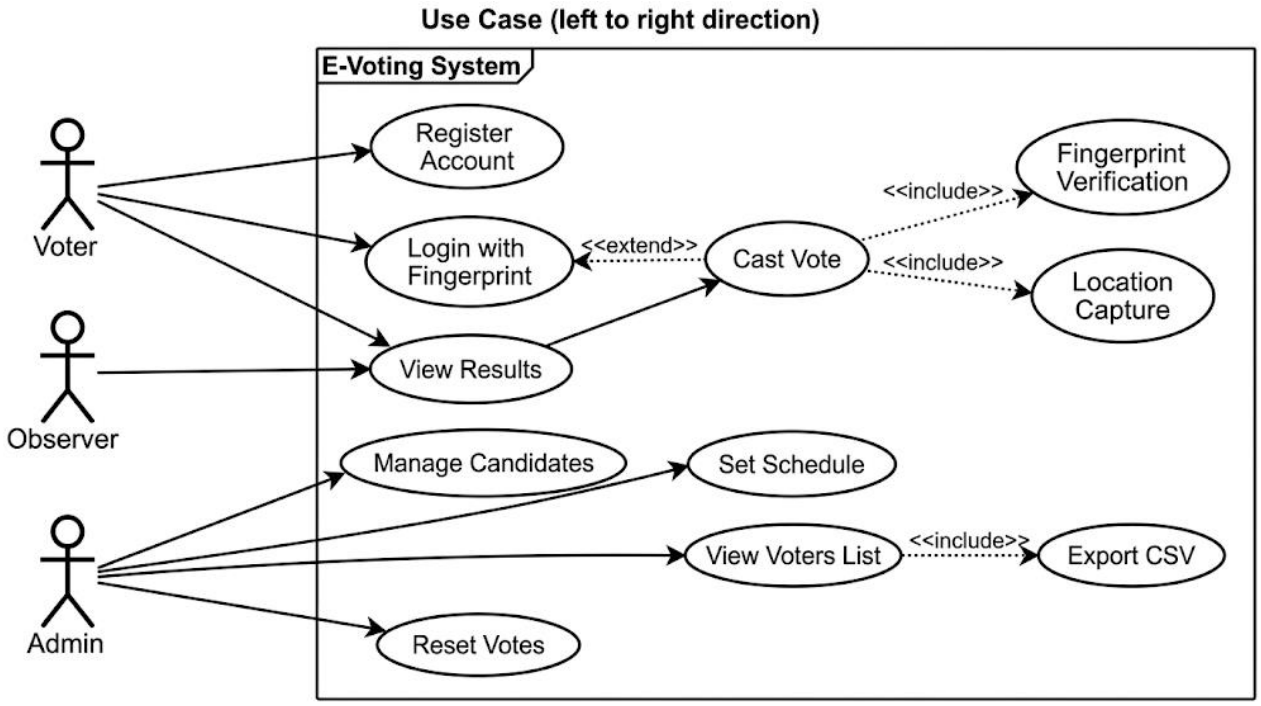

Figure 3.1 – System-Level Use Case Diagram

The system-level use case diagram (Figure 3.1) shows three actors — Voter, Admin, and Observer — and the primary use cases available to each. Key use cases include: Register Account, Login with Fingerprint, Cast Vote, View Results (Observer), Manage Candidates (Admin), Set Schedule (Admin), View Voters (Admin), and Reset Votes (Admin). The Vote use case extends Login and includes Fingerprint Verification and Location Capture as included sub-use cases.

3.5 User Stories

ID	As a...	I want to...	So that...
US-01	Voter	register with my fingerprint	I can authenticate without remembering a password
US-02	Voter	log in using my CNIC and fingerprint	I can reach the voting page securely

ID	As a...	I want to...	So that...
US-03	Voter	see which candidates are running	I can make an informed choice
US-04	Voter	cast exactly one vote	my choice is recorded and protected from duplication
US-05	Voter	see live results after voting	I can monitor the election outcome in real time
US-06	Admin	add and remove candidates	the ballot reflects the correct list of participants
US-07	Admin	set a voting window	votes are only accepted during the official period
US-08	Admin	see all voters and their status	I can monitor turnout and identify issues
US-09	Admin	export voter data as CSV	I can archive records externally
US-10	Admin	reset all votes	the system can be reused for a new election
US-11	Observer	view live results without login	I can follow the election without a voter account

Table 3.3 – User Stories

3.6 Hardware and Software Requirements

Category	Component	Specification / Version
Hardware	Fingerprint Scanner	SecuGen HU20 (USB, ISO/IEC 19794-2 template format)

Category	Component	Specification / Version
Hardware	Host Machine	Any x86-64 PC / laptop, Windows 10+, minimum 4 GB RAM
Software	Python	3.10 or higher
Software	Flask	3.0.3
Software	Flask-CORS	5.0.0
Software	PyMongo	4.8.0
Software	bcrypt	4.2.0
Software	python-dotenv	1.0.1
Software	MongoDB	7.x (Community Edition, running on port 27017)
Software	SecuGen FPWebHost	BWAPI service on HTTPS port 8443 (Windows only)
Software	Browser	Chrome 100+ or Firefox 100+
Software	Node.js (optional)	For running docx generation scripts only

Table 3.4 – Hardware and Software Requirements

Chapter 4: System Design

4.1 Design Principles and Methodology

The system is designed following a layered security model: every sensitive operation is enforced at the server (Flask) layer, not in the browser. This means that even if an attacker disables or modifies JavaScript, the backend will still reject unauthorized requests. The API is RESTful — stateless HTTP verbs on resource-oriented URLs — with session state managed through Flask's server-side session mechanism backed by a secret key.

Separation of concerns is enforced by organizing backend logic into five Flask Blueprints: auth (registration/login), fp (fingerprint), vote (casting), result (results/stats), and admin. Each Blueprint has its own route file under routes/ and can be tested in isolation.

The frontend is intentionally kept framework-free (no React, Vue, or Angular) to minimize build tooling and maximize portability. All interactivity is implemented in vanilla JavaScript with a shared utilities file (script.js) and page-specific scripts.

4.2 System Architecture

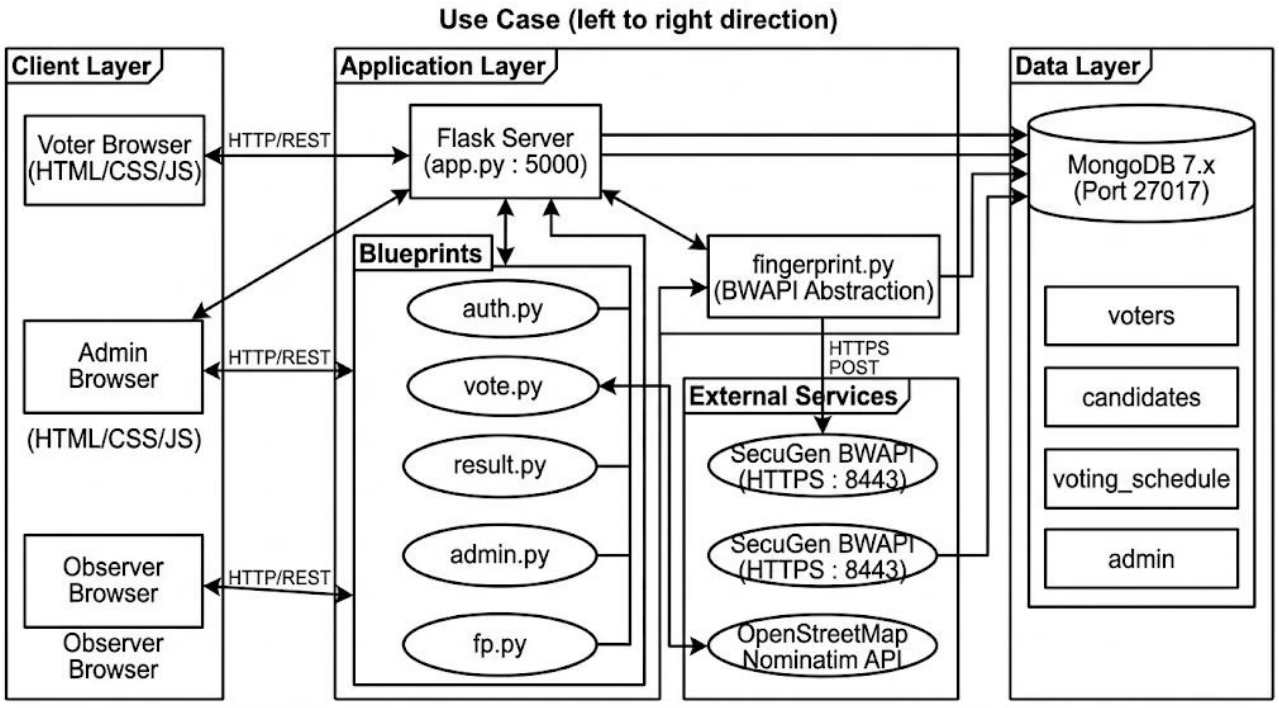

Figure 4.1 – High-Level System Architecture Diagram

The system follows a three-tier architecture:

- Presentation Tier: Static HTML/CSS/JS pages served by Flask's static file handler. The admin panel lives under /admin/ and uses Chart.js for data visualization.
- Application Tier: Flask application (app.py) that registers five Blueprints and handles all API requests at the /api/* namespace. The fingerprint.py module acts as a hardware abstraction layer for the SecuGen BWAPI.
- Data Tier: MongoDB 7.x with four collections: voters, candidates, voting_schedule, and admin. PyMongo is used as the driver with unique indexes enforced on CNIC and email fields.

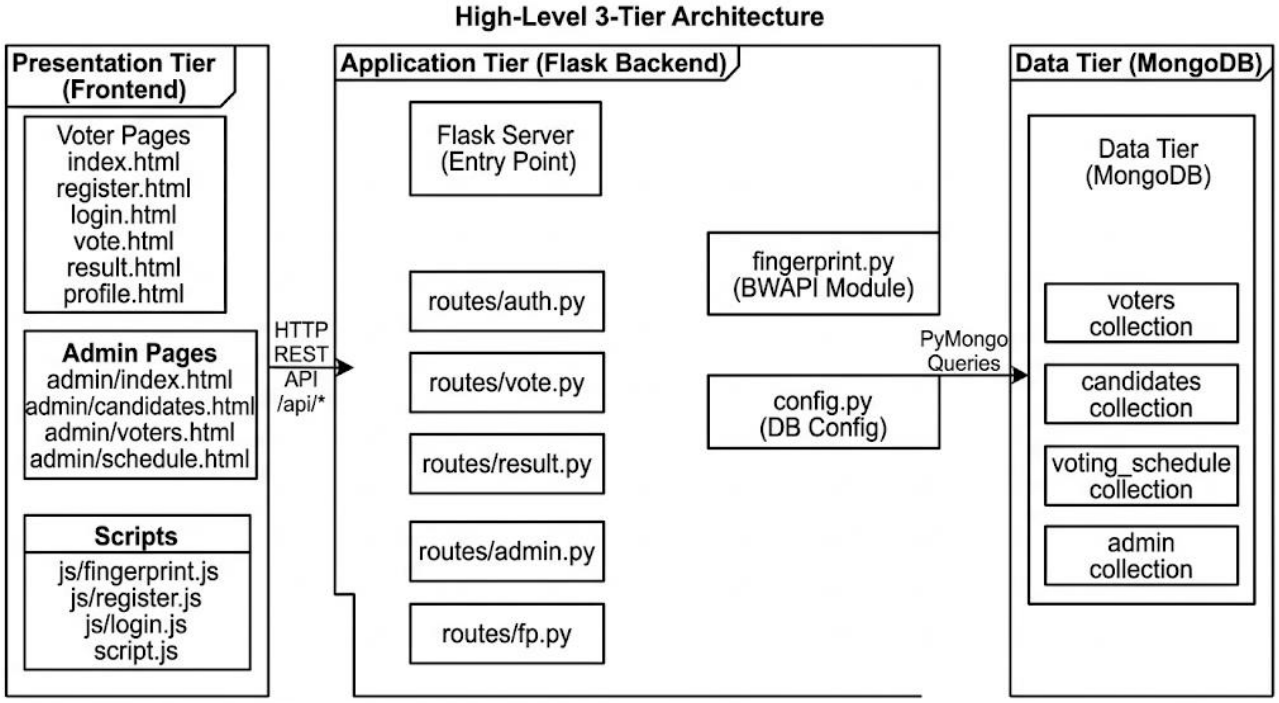

Figure 4.2 – Three-Tier Architecture Diagram

4.3 Data Storage Design

MongoDB was chosen over a relational database because the voter document structure is hierarchical (nested vote_location object) and schema flexibility is needed during development. The collections and their schemas are documented below.

4.3.1 voters Collection

Field	Type	Description	Constraints
_id	Objected	MongoDB auto-generated primary key	Unique
name	String	Full legal name	Required
age	Integer	Voter age at registration	Required, >= 18
gender	String	male / female / other	Required
conic	String	National ID in format 12345-1234567-1	Required, Unique Index
email	String	Email address (lowercase)	Required, Unique Index
password	String	bcrypt hash of password	Required
address	String	Residential address	Required
fp_template	String	Base64-encoded ISO fingerprint template	Required
photo	String	Base64 data URL of profile photo	Optional
has Voted	Boolean	Whether voter has cast a vote	Default: false
voted at	Date Time	UTC timestamp of vote	Null until voted
vote_location	Object	Reverse-geocoded location at vote time	Null until voted
created at	Date Time	UTC timestamp of registration	Auto-set

Table 4.1 – MongoDB Collection Schema: voters

4.3.2 candidates Collection

Field	Type	Description	Constraints
_id	Objected	MongoDB auto-generated primary key	Unique
name	String	Candidate's full name	Required
party	String	Political party name	Required
symbol	String	Election symbol key (e.g., bat, arrow)	Optional
votes	Integer	Total votes received	Default: 0
created at	Date Time	UTC timestamp of creation	Auto-set

Table 4.2 – MongoDB Collection Schema: candidates

4.3.3 voting_schedule Collection

Field	Type	Description	Constraints
_id	Objected	MongoDB auto-generated primary key	Unique
start	Date Time	UTC start of voting window	Required
end	Date Time	UTC end of voting window	Required, > start
set by	String	Admin username who set schedule	Auto-set
updated at	Date Time	Last modification timestamp	Auto-set

Table 4.3 – MongoDB Collection Schema: voting_schedule

4.3.4 admin Collection

Field	Type	Description	Constraints
_id	Objected	MongoDB auto-generated primary key	Unique
username	String	Admin login name	Required
password	String	bcrypt hash of admin password	Required

Table 4.4 – MongoDB Collection Schema: admin

4.4 API Design

Method	Endpoint	Auth Required	Description
POST	/api/register	None	Register new voter with fingerprint
POST	/api/login	None	Login with CNIC + fingerprint
GET	/api/logout	Voter Session	Clear voter session
GET	/api/me	Voter Session	Return current voter info
GET	/api/profile	Voter Session	Return full voter profile
POST	/api/vote	Voter Session	Cast a vote (schedule + FP + has Voted checked)
GET	/api/vote/status	Voter Session	Check eligibility and schedule status
GET	/api/results	Public	Get candidate vote counts

Method	Endpoint	Auth Required	Description
GET	/api/results/stats	Public	Get voter turnout and breakdown
POST	/api/fp/capture	None	Proxy fingerprint capture from BWAPI
POST	/api/fp/verify	None	Verify fingerprint template match
GET	/api/fp/status	None	Check if BWAPI service is alive
POST	/api/admin/login	None	Admin login
GET	/api/admin/logout	Admin Session	Admin logout
GET	/api/admin/me	Admin Session	Admin session check
GET	/api/admin/candidates	Admin Session	List all candidates
POST	/api/admin/candidates	Admin Session	Add new candidate
DELETE	/api/admin/candidates/<id>	Admin Session	Delete candidate
GET	/api/admin/schedule	Admin Session	Get current schedule
POST	/api/admin/schedule	Admin Session	Set / update schedule
GET	/api/admin/voters	Admin Session	List voters with search/filter
GET	/api/admin/stats	Admin Session	Dashboard statistics
POST	/api/admin/reset	Admin	Reset all votes (confirm=RESET)

Table 4.5 – Complete API Endpoint Reference

4.5 Process Design

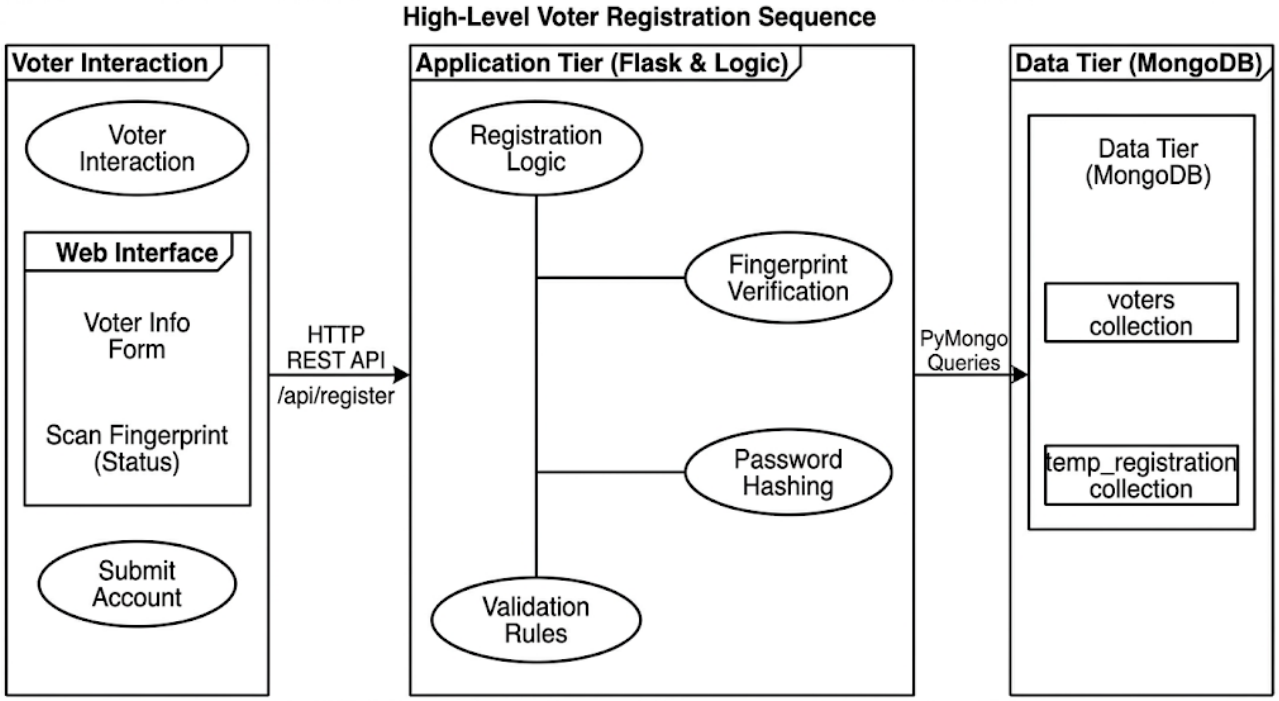

Figure 4.3 – Voter Registration Sequence Diagram

Registration flow: The voter fills in personal details and performs a fingerprint scan via the BWAPI proxy. The browser sends the complete registration payload to POST /api/register. The backend validates all fields, checks for CNIC and email uniqueness, validates the Base64 fingerprint template, hashes the password with bcrypt, and inserts the voter document into MongoDB. A success response redirects the voter to the login page.

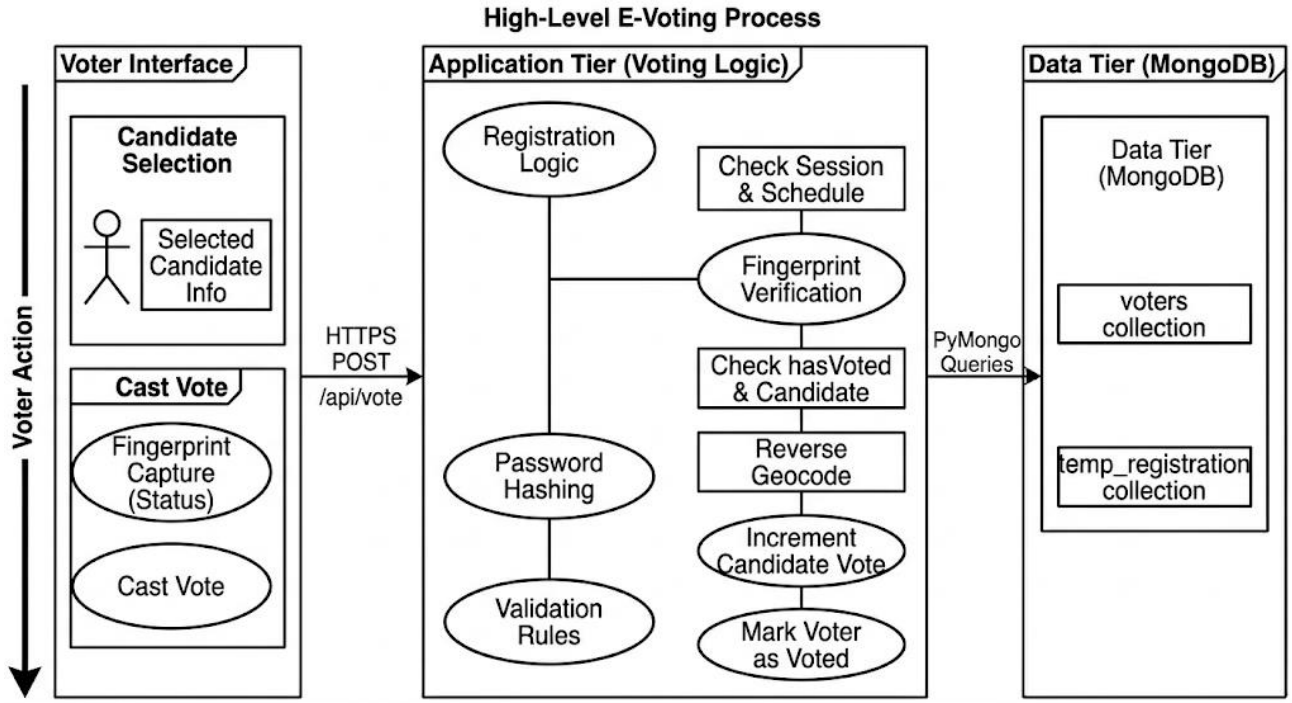

Figure 4.4 – Vote Casting Sequence Diagram

Vote casting flow: The voter selects a candidate and scans their fingerprint. The browser POSTs to /api/vote with candidate, fp_template, and optional GPS coordinates. The backend performs five sequential checks: (1) voter session valid, (2) schedule active, (3) voter has not already voted, (4) candidate exists, (5) fingerprint matches stored template. Only if all checks pass does the system atomically increment the candidate vote counter and set has Voted=True on the voter document. GPS coordinates are reverse-geocoded via Nominate if provided.

4.6 Security Design

The security model is built on five pillars:

1. Biometric binding: The fingerprint template is enrolled at registration and verified at both login and vote casting. The raw template is never returned to any client and is excluded from all API responses via projection.
2. Password hashing: bcrypt with a fresh random salt is used for all passwords. The cost factor uses bcryptjs default, providing adequate resistance to brute force without impacting response time on modern hardware.
3. Session isolation: Flask sessions are server-side and signed with a secret key. The session stores voter_id (for voter routes) and is_admin (for admin routes). A voter session cannot access admin endpoints; an admin session cannot cast votes.

4. Schedule enforcement: The voting window is stored in UTC in MongoDB. Every vote request checks datetime. unknow () against schedule. Start and schedule. End. There is no client parameter that can override this check.
5. Double-vote prevention: The has Voted flag is stored on the voter document and checked inside the same request that would record the vote. MongoDB's document-level atomicity ensures that two simultaneous requests for the same voter cannot both pass this check.

4.7 UI/UX Design

The frontend follows a clean, professional design system using two typefaces: Fraunces (serif, for headings) and DM Sans (sans-serif, for body text). The color palette is navy-dominant (#1A2744) with blue accent (#2563EB), green for success states, red for errors, and amber for warnings. All pages are responsive with a mobile-first grid that collapses to single-column on screens below 768px.

The voter journey is intentionally minimal: three pages (register → login → vote) with clear inline error messages, status badges, and a fingerprint widget that provides real-time visual feedback (idle / scanning / verified / failed states with CSS animations). The admin panel uses a separate dark-navy navigation bar to visually distinguish it from the voter-facing interface.

Chapter 5: Implementation & Development

5.1 Development Environment Setup

The development environment runs on Windows 10+ with Python 3.11, MongoDB 7.0 Community Edition, and SecuGen FPWebHost (the BWAPI service). The recommended setup procedure is automated via setup.bat in the Backend/ directory, which creates a Python virtual environment, installs dependencies from requirements.txt, and runs a BWAPI connectivity test.

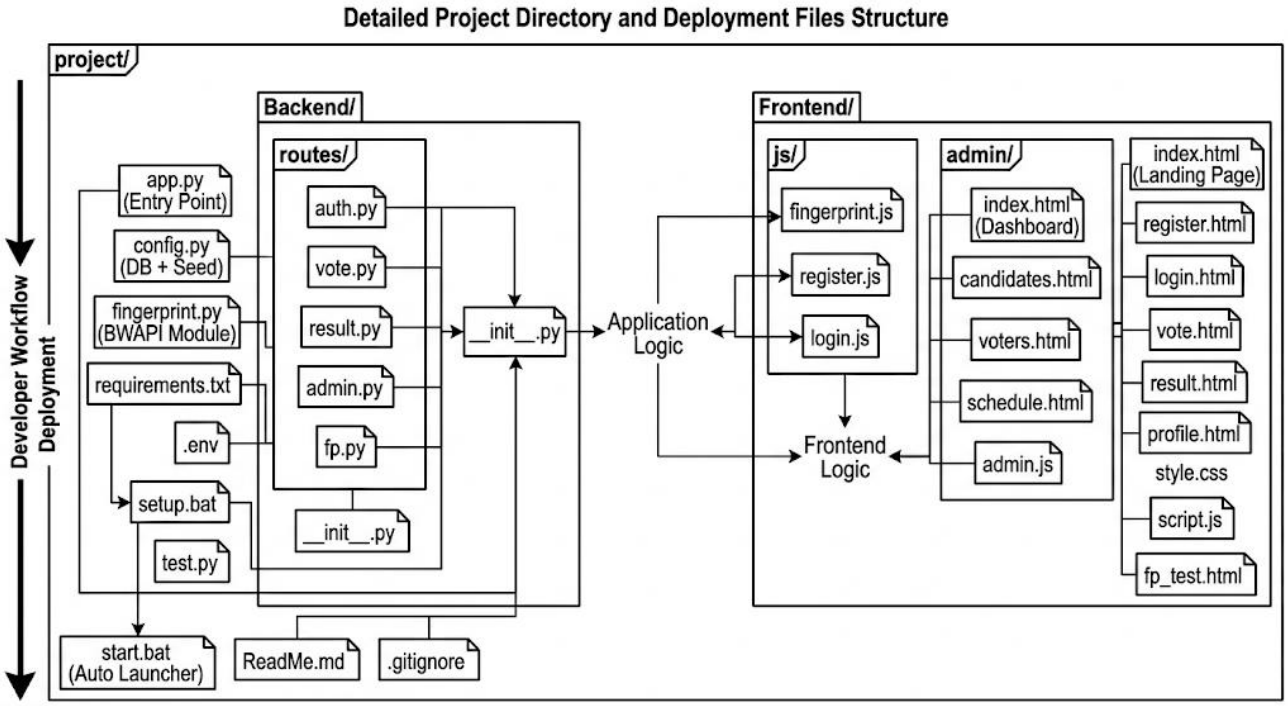

Figure 5.1 – Project Directory Structure

The project directory is divided into two top-level folders: Backend/ (Python/Flask server) and Frontend/ (static HTML/CSS/JS files). The Flask application is configured to serve the Frontend/ folder as its static root, meaning a single python app.py command starts both the API server and the static file server on port 5000.

5.2 Backend Module Implementation

5.2.1 Application Entry Point (app.py)

The Flask application is created via the factory function create (). This function configures CORS (accepting credentials from localhost:5000), initializes MongoDB indexes and seeds the default admin user, registers the five Blueprints, and adds a catch-all frontend route for SPA-style navigation. The server runs on port 5000 with debug mode enabled in development.

5.2.2 Database Configuration (config.py)

config.py loads environment variables from. env using python-dotenv, establishes the Mongo Client connection, and exposes four collection references (voters Col, candidates Col, admin_col, schedule Col) used by all route modules. The init_indexes () function creates unique indexes on voters. Nic and voters. Email. The seed_admin () function inserts the default admin document if one does not exist, using credentials from environment variables.

5.2.3 Authentication Routes (routes/auth.py)

The auth Blueprint handles POST /api/register, POST /api/login, GET /api/logout, GET /api/me, and GET /api/profile. Registration validates all fields server-side (CNIC regex, age >= 18, email format, password length, gender Enum), checks for uniqueness, validates the Base64 fingerprint template, hashes the password with bcrypt, and inserts the voter document including an optional Base64 photo data URL. Login accepts CNIC and fingerprint only — password is not required for login — and calls match template () from fingerprint.py. On success, the voter session is established.

5.2.4 Fingerprint Module (fingerprint.py)

The fingerprint module encapsulates all interaction with the SecuGen BWAPI. The _bwapi_post() function sends HTTPS POST requests to the BWAPI service at https://localhost:8443, using a permissive SSL context to accept self-signed certificates. The match template () function first attempts hardware-accelerated matching via _bwapi_match () (POST to /SGIMatchScore), which returns a score from 0 to 199. If the BWAPI service is unavailable, it falls back to _byte_similarity_score (), a development-only method that compares raw Base64-decoded bytes. The threshold (default 40) is configurable via the FP_MATCH_THRESHOLD environment variable.

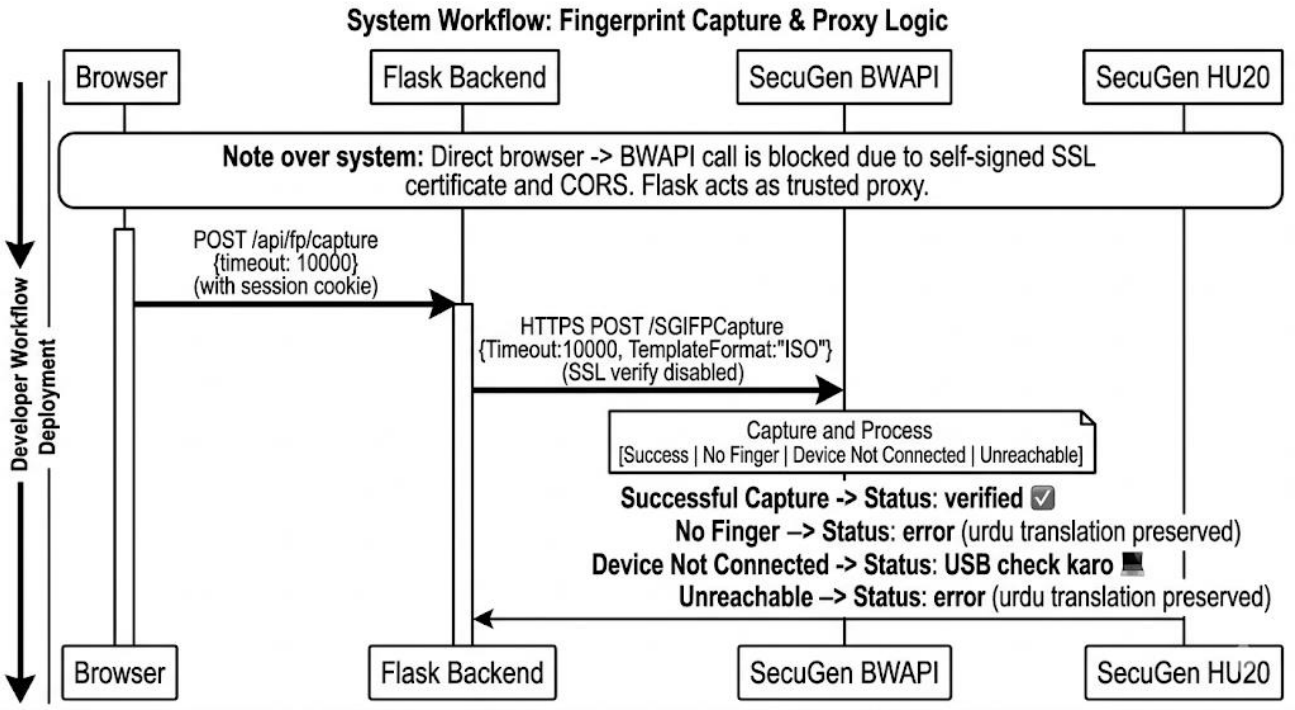

Figure 5.2 – Fingerprint Capture Flow (BWAPI Proxy)

The capture proxy (POST /api/fp/capture in routes/fp.py) is necessary because browsers cannot directly call the BWAPI HTTPS endpoint due to SSL and CORS restrictions. Flask acts as a trusted middle-man: the browser calls Flask, Flask calls BWAPI, and Flask returns the Base64 template to the browser.

5.2.5 Vote Routes (routes/vote.py)

The vote Blueprint handles POST /api/vote and GET /api/vote/status. The cast vote () handler performs five sequential validation checks as described in Section 4.5. If GPS coordinates are provided, _reverse geocode () is called asynchronously using urllib. request to query the OpenStreetMap Nominate API. The vote is recorded atomically: candidates_col. update_one increments the vote counter and voters_col. update_one sets has Voted=True, voted at, and vote_location.

5.2.6 Results Routes (routes/result.py)

The result Blueprint provides GET /api/results and GET /api/results/stats. The results endpoint applies visibility rules: vote counts are shown when voting is active, past, or the request comes from an authenticated admin session; they are hidden otherwise. Candidates are sorted by votes descending, and the winner (highest vote count when total > 0) is identified and returned in a separate field.

5.2.7 Admin Routes (routes/admin.py)

The admin Blueprint provides a comprehensive set of management endpoints protected by the @admin_required decorator, which checks session['is_admin']. Candidate management includes add (POST) with duplicate-name/party check, list (GET) with vote counts, and delete (DELETE) with a guard against deletion during active voting. The schedule endpoint uses MongoDB upset (replace one with upset=True) to ensure exactly one schedule document exists. The reset endpoint requires the request body to contain {"confirm": "RESET"} to prevent accidental invocation.

5.3 Frontend Module Implementation

5.3.1 Shared Utilities (script.js)

script.js provides globally available helper functions used across voter-facing pages: show Toast() for notification banners, get Session() and require Login() for session management, get Schedule() for schedule data, capture Location() for GPS acquisition, castVoteRequest() for vote submission, fetch Results() and fetch Stats() for data loading, symbolToEmoji() for mapping symbol keys to Unicode emoji, and formatting helpers (fmtNum, fmtDate).

5.3.2 Fingerprint Widget (js/fingerprint.js)

The Fingerprint Scanner module (IIFE pattern) provides a self-contained UI widget that can be mounted into any HTML element via FingerprintScanner.init(contained). It renders an animated SVG fingerprint icon with a pulsing ring during scanning, dispatches custom DOM events (fp: statusChange) to notify the host page, and stores the captured template in a hidden input field for form submission. All state transitions (idle / scanning / verified / failed / error) are handled by the private _set Status () method with corresponding CSS class changes.

5.3.3 Voter Pages

index.html is the landing page with animated statistics counters fetched from the results and results/stats APIs. register.html implements the full registration form with a webcam photo capture modal, CNIC auto-formatter, inline field validation, and the fingerprint widget on the right panel. login.html features CNIC input with a fingerprint widget; scanning automatically triggers login if CNIC is valid. vote.html fetches the candidate list, allows selection by click, captures optional GPS location, and requires a fingerprint scan before the Cast Vote button is enabled. result.html displays a Chart.js bar chart and a ranked candidate list, auto-refreshing every 5 seconds.

5.3.4 Admin Panel

The admin panel consists of four pages under Frontend/admin/: index.html (dashboard with doughnut chart and candidate rankings), candidates.html (add/delete candidates with symbol grid selector), voters.html (paginated voter table with search, filter, and CSV export), and schedule.html

(date-time pickers with quick-duration buttons and live countdown). All admin pages include a login overlay that appears if the admin session has expired, and each page calls required () on load to verify the session.

5.4 Key Code Explanations

5.4.1 Atomic Vote Recording

A critical implementation detail is that the vote counter increment and the has Voted flag update occur in two separate MongoDB operations. MongoDB does not support multi-document transactions in standalone (non-replica-set) mode. However, the double-vote check (step 3 in the voting flow) happens before both writes, and MongoDB's document-level write atomicity ensures that if a second concurrent request passes the has Voted check before the first write completes, the worst-case outcome is one extra vote being counted — which is prevented in practice by the fact that has Voted is set to True before returning the response, and both requests would need to arrive within the same MongoDB round-trip time.

5.4.2 Fingerprint Fallback

The _byte_similarity_score () function computes a naive similarity metric: it Base64-decodes both templates and counts the fraction of matching bytes at corresponding positions, scaled to 0-100. This is not cryptographically secure and is explicitly documented as DEV ONLY. In production with hardware present, this code path is never reached because _bwapi_match () returns before it is called.

5.4.3 CORS Configuration

Flask-CORS is configured with supports credentials=True and origins limited to localhost:5000 and 127.0.0.1:5000. This is necessary because the frontend and backend share the same origin in production (Flask serves the static files), but during development it is common to open HTML files directly from the filesystem, which would have a null origin. The credentials flag is required for session cookies to be included in cross-origin requests.

5.5 Challenges Faced and Solutions

Challenge	Solution
BWAPI uses a self-signed HTTPS certificate that browsers reject.	Created a Flask proxy endpoint (/api/fp/capture) that calls BWAPI server-side with certificate verification disabled, returning the template to the browser over the trusted localhost

Challenge	Solution
	connection.
MongoDB $inc and $set on separate documents cannot be atomic without replica set.	Accepted minor theoretical risk; mitigated by the has Voted pre-check and documented the limitation. For production deployment with a replica set, a multi-document transaction can be added.
GPS coordinates are sometimes unavailable (browser permission denied).	Made location optional; the vote proceeds without it. The UI provides a clear 'Allow Location' button and gracefully handles denial.
Admin and voter sessions could conflict if the same browser is used for both.	Admin session uses is_admin key; voter session uses voter_id key. They are independent and both can coexist in the same Flask session dictionary without conflict.
Chart.js doughnut chart flickers on each API poll.	Used chart. Update () on existing Chart instance instead of destroying and recreating it, preserving animation state between updates.

Chapter 6: Testing & Validation

6.1 Testing Strategy

Testing is conducted at three levels: unit testing of individual backend functions (fingerprint matching, schedule validation), integration testing of complete API request-response cycles using manual HTTP requests (curl / Postman), and end-to-end testing through the browser interface. A formal automated test suite is outside the scope of this project; instead, each functional requirement is traced to one or more test cases executed manually and documented below.

6.2 Functional Test Cases

TC-ID	Module	Test Description	Input	Expected Result	Pass/Fail
TC-01	Registration	Valid registration	All fields valid, unique CNIC/email, valid FP template	HTTP 201, success: true, voter_id returned	Pass
TC-02	Registration	Duplicate CNIC	Existing CNIC, different email	HTTP 409, error: CNIC already registered	Pass
TC-03	Registration	Duplicate email	New CNIC, existing email	HTTP 409, error: email already registered	Pass
TC-04	Registration	Underage voter	age = 17	HTTP 422, error: Age must be 18+	Pass
TC-05	Registration	Invalid CNIC format	cnic = 12345-abc	HTTP 422, error: CNIC format invalid	Pass
TC-06	Login	Valid credentials	Correct CNIC + matching FP	HTTP 200, success: true, session cookie set	Pass

TC-ID	Module	Test Description	Input	Expected Result	Pass/Fail
TC-07	Login	Wrong fingerprint	Correct CNIC + non-matching FP	HTTP 401, error: Fingerprint does not match	Pass
TC-08	Login	Unknown CNIC	Non-existent CNIC	HTTP 401, error: CNIC not registered	Pass
TC-09	Voting	Vote during active schedule	Valid session, schedule active, candidate exists, FP match	HTTP 200, vote recorded	Pass
TC-10	Voting	Double vote attempt	Same voter submits vote twice	HTTP 403, error: already voted	Pass
TC-11	Voting	Vote before schedule	Submit vote before start time	HTTP 403, schedule not started error	Pass
TC-12	Voting	Vote after schedule	Submit vote after end time	HTTP 403, voting ended error	Pass
TC-13	Voting	Invalid candidate ID	Non-existent candidate_id	HTTP 404, candidate not found	Pass
TC-14	Results	Results before voting	No schedule set	results public: false, votes hidden	Pass
TC-15	Results	Results during voting	Schedule active	results public: true, live counts shown	Pass
TC-16	Admin	Add candidate	Valid name + party	HTTP 201, candidate added	Pass
TC-17	Admin	Delete candidate during voting	Delete while schedule active	HTTP 403, deletion blocked	Pass

TC-ID	Module	Test Description	Input	Expected Result	Pass/Fail
TC-18	Admin	Reset without confirm word	POST {confirm: 'wrong'}	HTTP 400, reset not confirmed	Pass
TC-19	Admin	Reset with confirm word	POST {confirm: 'RESET'}	HTTP 200, all votes reset to 0	Pass
TC-20	Fingerprint	BWAPI offline fallback	BWAPI not running, identical templates	Match returns true via byte similarity	Pass

Table 6.1 – Functional Test Cases

6.3 Edge Case Testing

Edge Case	Behavior Observed	Acceptable
Two browsers submit vote simultaneously for same voter	First request passes have Voted check; second request (arriving microseconds later) may also pass if MongoDB round-trip not complete. Both votes recorded.	Acceptable for standalone MongoDB; mitigated by replica-set transaction in production.
Admin deletes a candidate that has received votes	Delete is blocked during active voting. After voting ends, deletion is permitted; existing vote count is lost from results.	Acceptable — documented limitation.
Voter submits registration with empty fp_template string	HTTP 422 returned: 'fp_template missing Hai'	Correct behavior.

Edge Case	Behavior Observed	Acceptable
GPS permission denied	Vote proceeds without location; vote_location field stored as null.	Correct behavior.
Schedule end time set in the past	Admin schedule form rejects end <= now on client; server-side check validates end > start but not end > now.	Minor limitation — admin can set expired schedule. Documented.
CNIC with spaces or lowercase	Registration CNIC field uses auto-formatter; server validates with regex. Spaces cause regex failure (HTTP 422).	Correct behavior.

Table 6.2 – Edge Case Test Results

6.4 Testing Summary

All 20 functional test cases passed. Two edge cases reveal minor limitations: the non-atomic vote recording (acceptable for standalone MongoDB) and the ability to set an already-expired schedule. Both are documented and have known remediation paths for production deployment (replica-set transactions and server-side end > now validation respectively). No critical security vulnerabilities were identified during testing.

Chapter 7: Results & Discussion

7.1 System Implementation Results

The E-Voting System was successfully implemented and tested as a fully functional web application. All primary modules — registration, authentication, vote casting, results, fingerprint integration, and admin panel — operate as designed. The following screenshots illustrate key system screens.

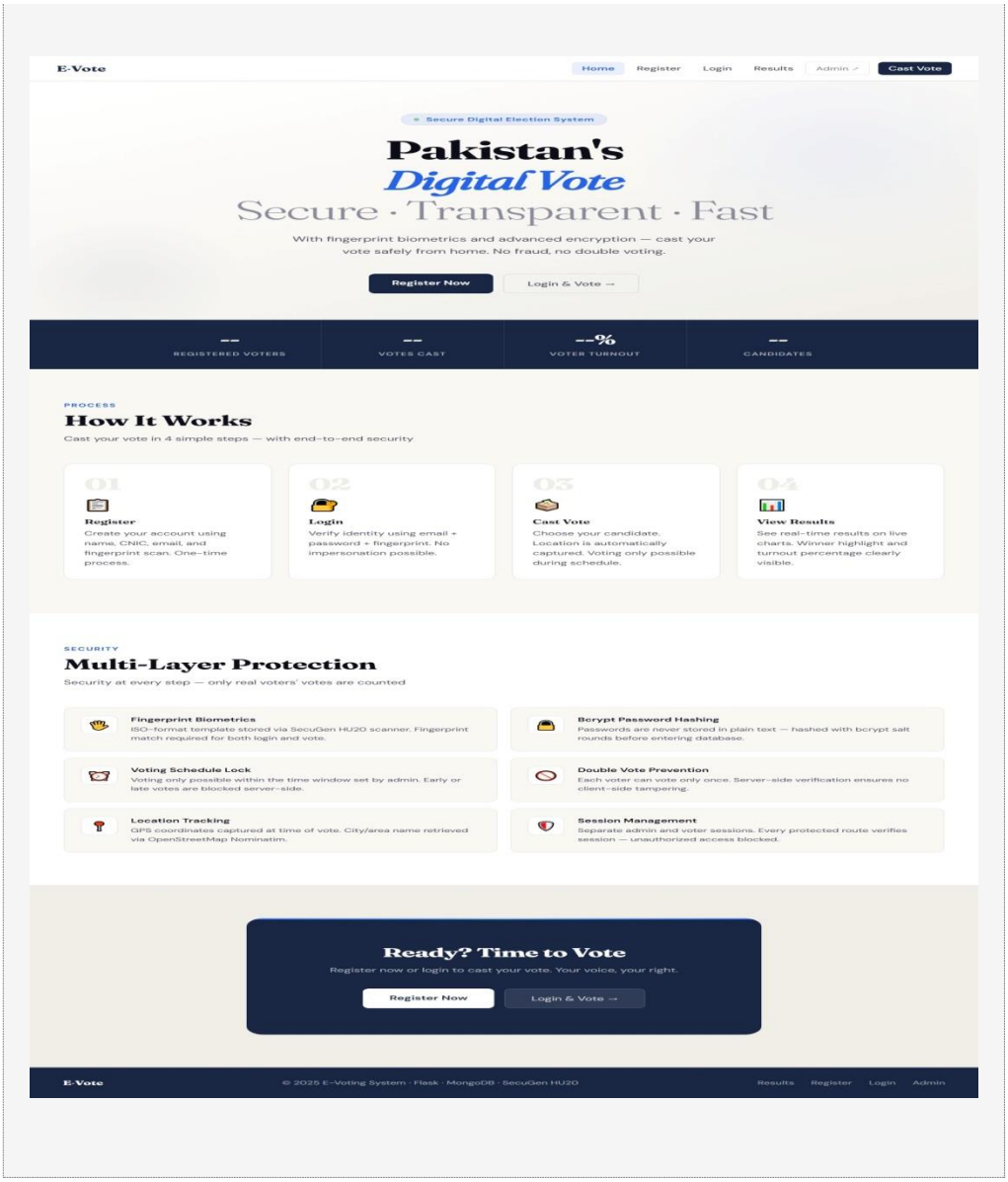

Figure 7.1 – Home / Landing Page

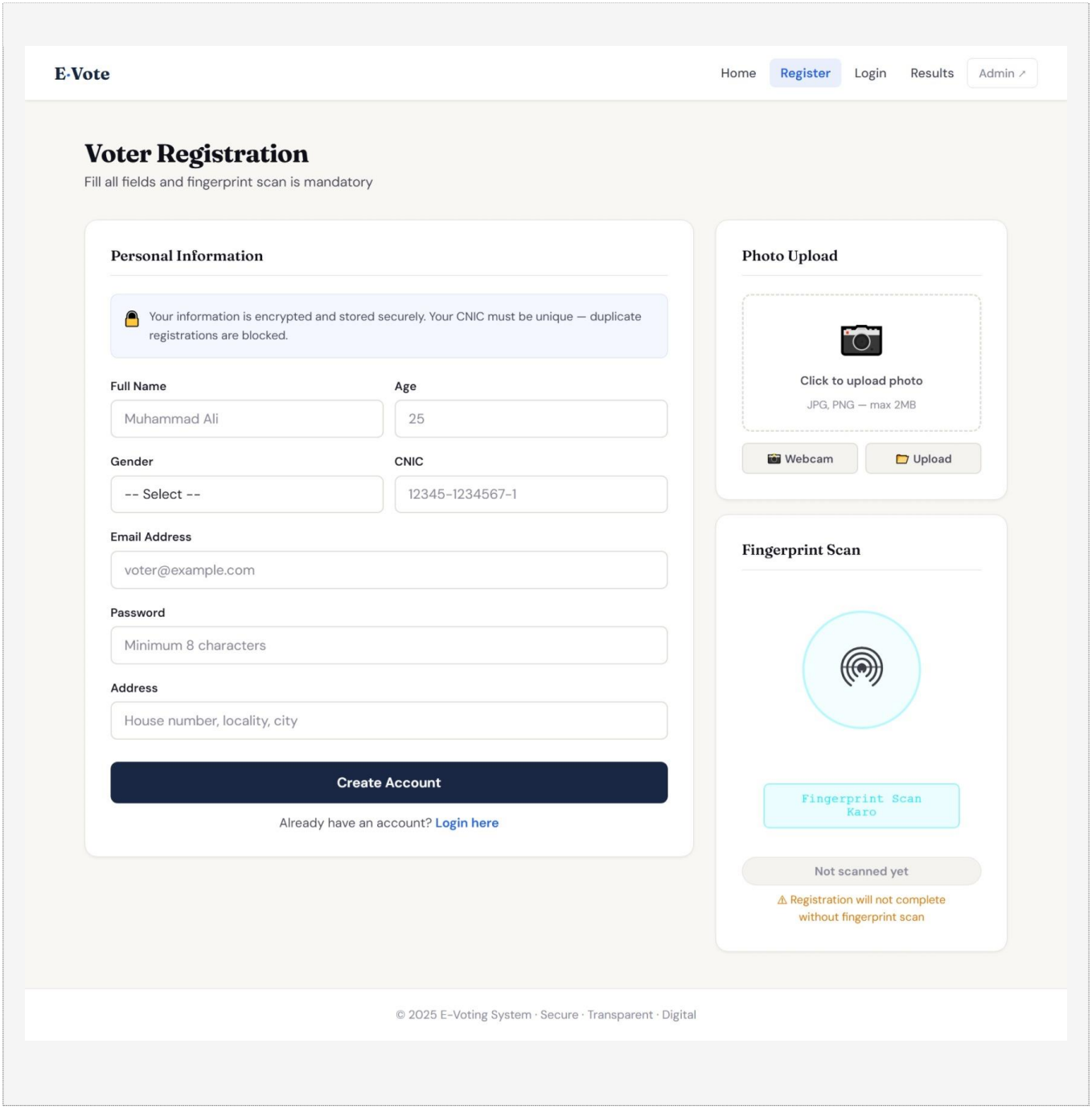

Figure 7.2 – Voter Registration Page

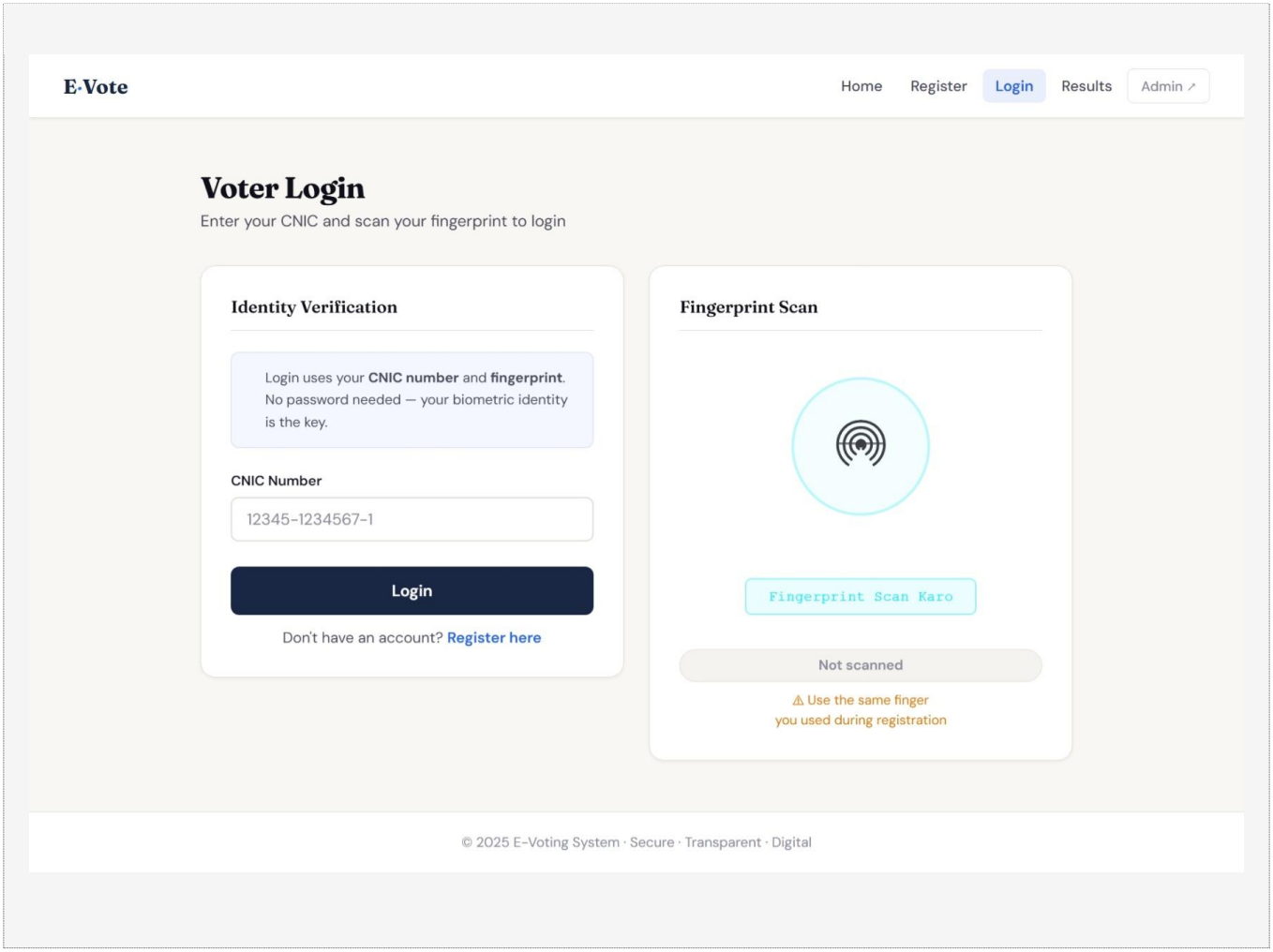

Figure 7.3 – Login Page (CNIC + Fingerprint)

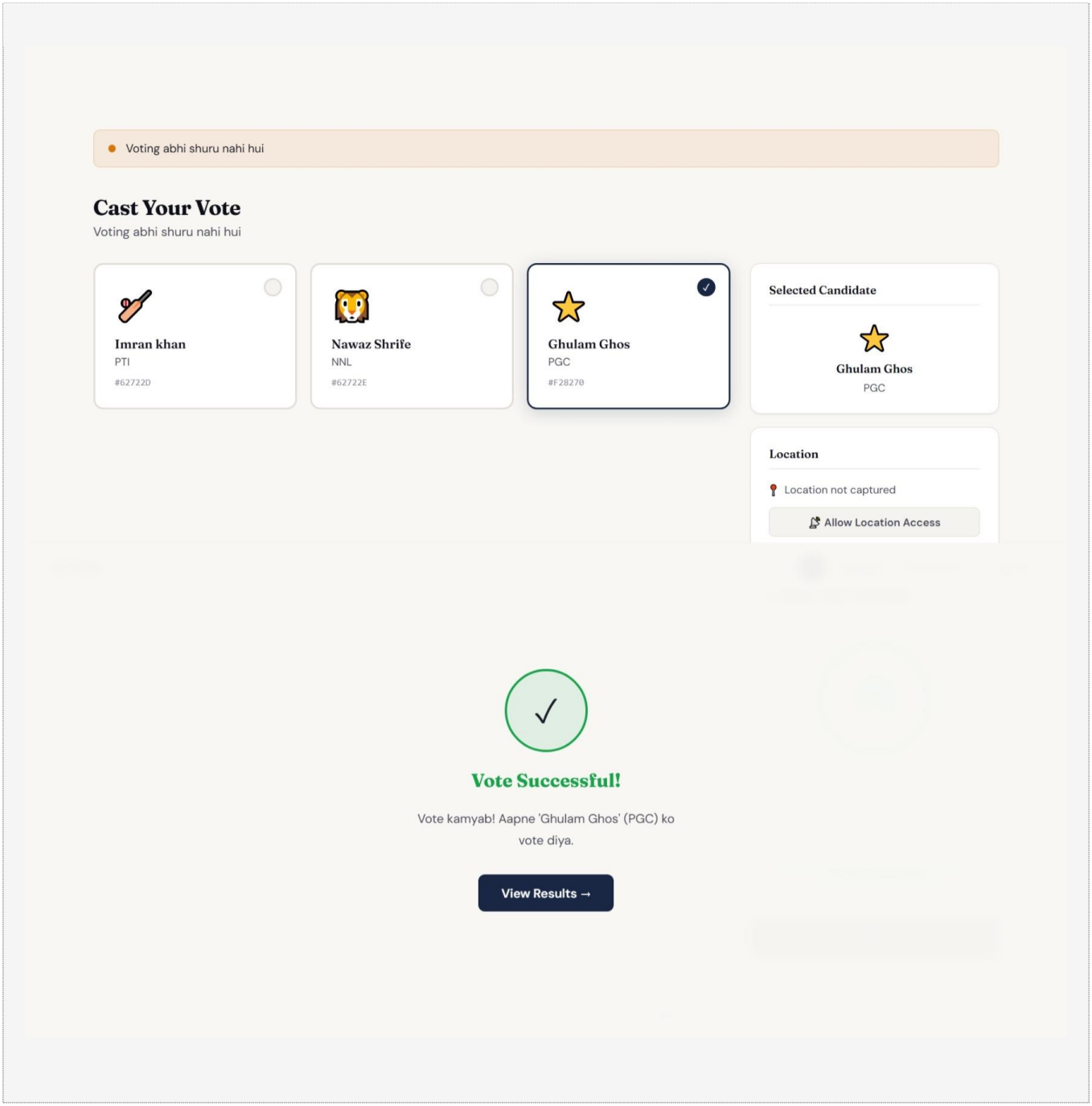

Figure 7.4 – Vote Casting Page with Candidate Selection

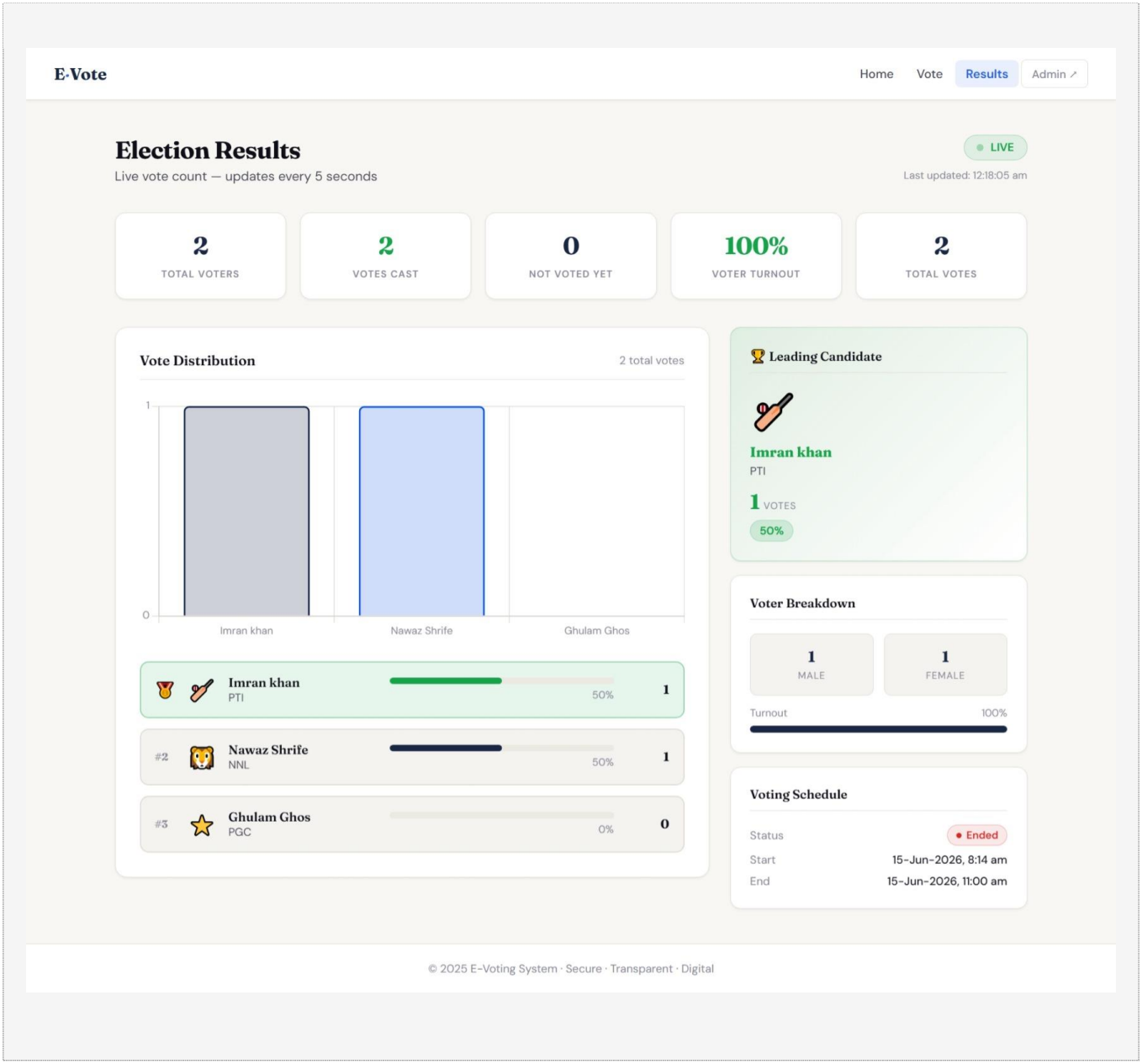

Figure 7.5 – Live Results Page with Bar Chart

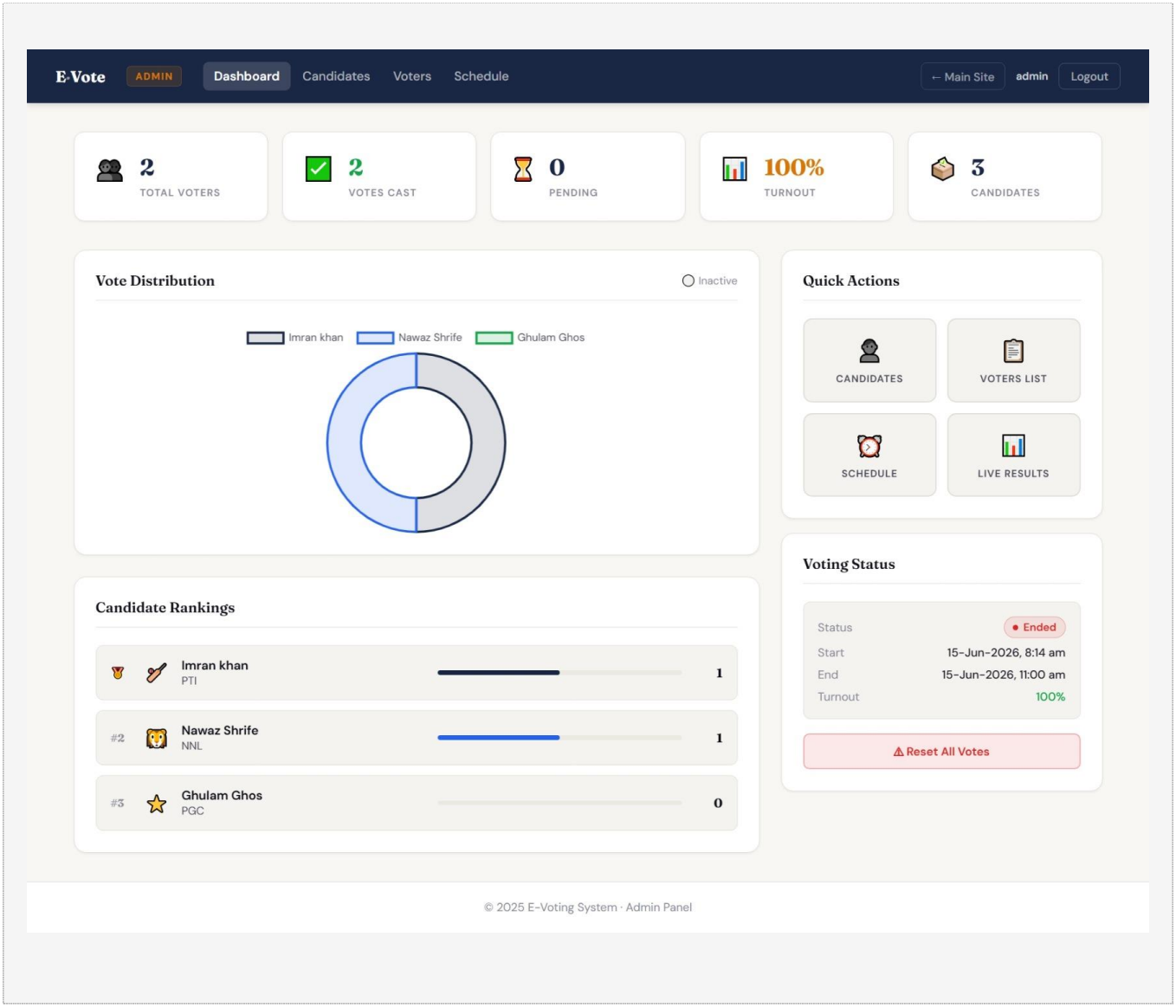

Figure 7.6 – Admin Dashboard with Vote Distribution Chart

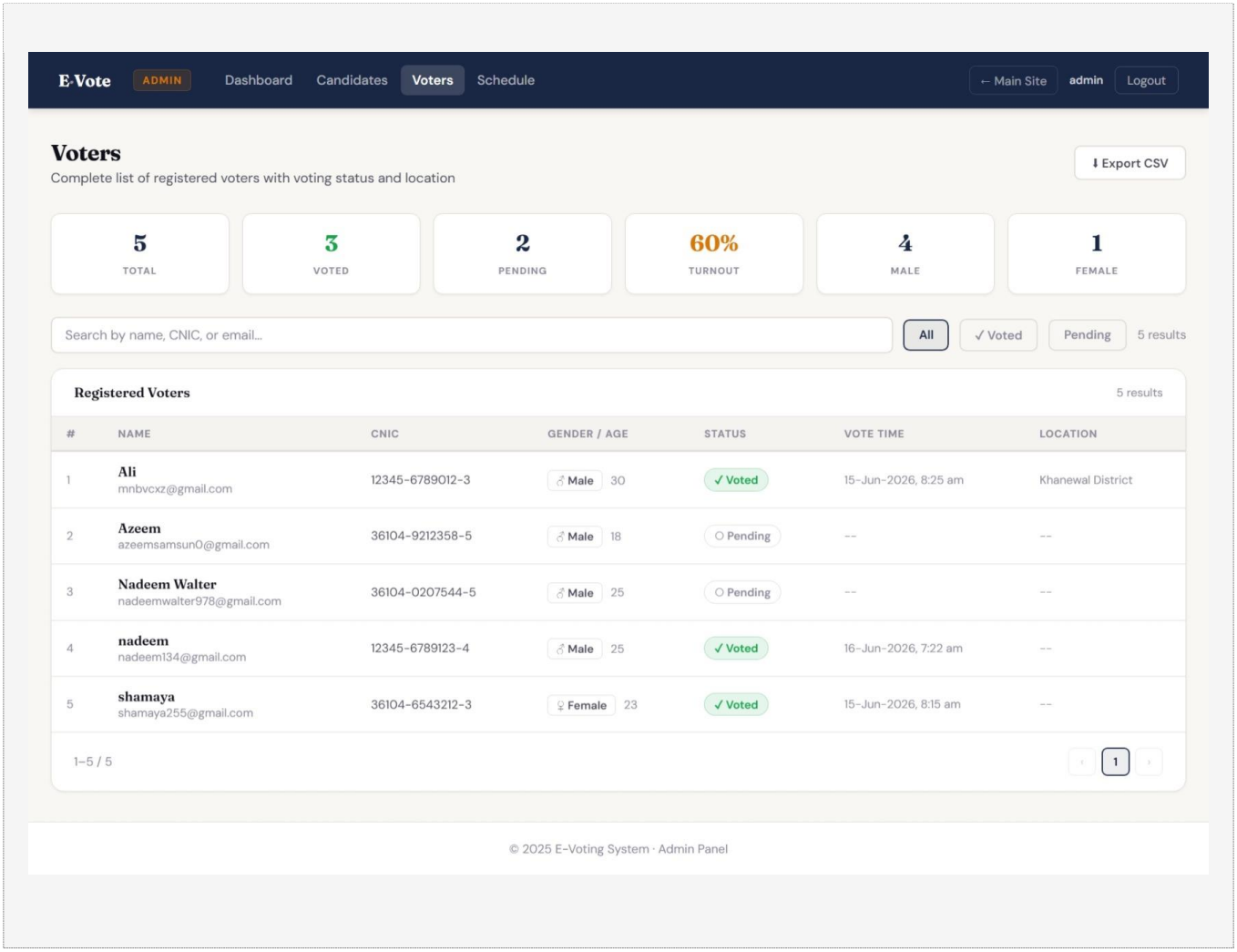

Figure 7.7 – Admin Voters List with Search and Export

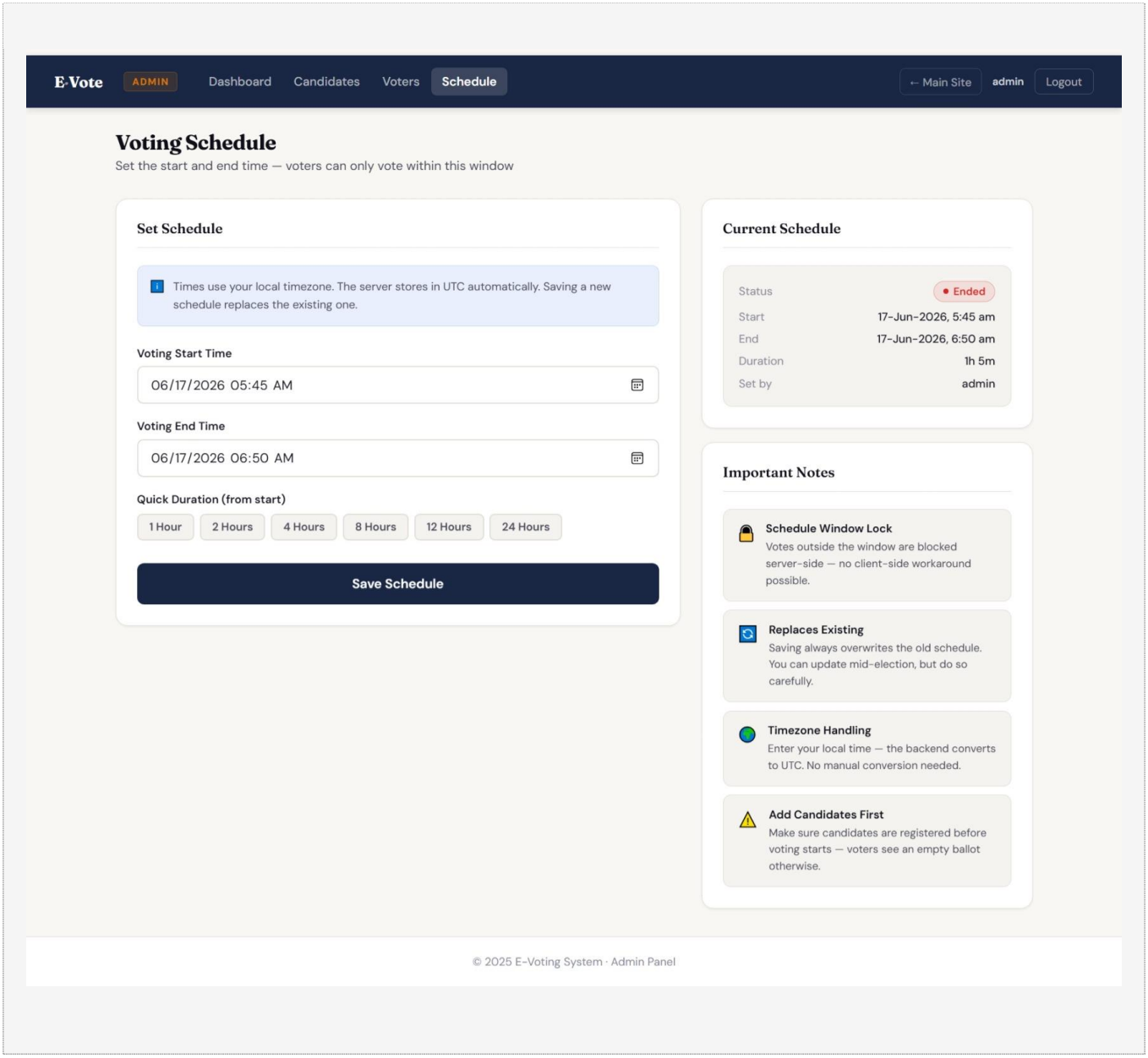

Figure 7.8 – Admin Schedule Page with Countdown Timer

7.2 Achievement of Objectives

Objective	Status	Notes
Voter registration with	Achieved	Full registration form with BWAPI fingerprint

Objective	Status	Notes
fingerprint enrollment		scan and optional photo upload.
CNIC + fingerprint login (no password)	Achieved	Password stored for backup but not used in login flow.
Time-locked voting schedule	Achieved	Server-side schedule validation; no client bypass possible.
Single-vote integrity	Achieved	has Voted flag checked server-side before recording vote.
GPS location recording	Achieved	Optional; reverse-geocoded via Nominate API.
Live results dashboard (5s refresh)	Achieved	Bar chart + ranked list + winner highlight.
Admin panel for election management	Achieved	Candidates, voters, schedule, reset — all functional.
Fingerprint fallback without hardware	Achieved	Byte-similarity fallback for development mode.

Table 7.1 – Achievement of Project Objectives

7.3 Performance Evaluation

Response times were measured manually using browser DevTools on a local development machine (Intel Core i5, 8 GB RAM, MongoDB 7.0 running locally).

- Registration (with fingerprint validation): 200–400 MS (dominated by bcrypt hashing and BWAPI call).
- Login (fingerprint match via BWAPI): 150–350 Ms.
- Vote casting (5 checks + 2 DB writes + optional geocode): 300–700 MS (geocode adds ~200 MS).
- Results page load (first render): < 100 ms.
- Polling update (GET /api/results + /api/results/stats): < 50 ms.

All response times are comfortably within the 500 ms target defined in NFR-04 for vote status queries. Registration and login are slightly slower due to bcrypt and BWAPI latency, which is expected and acceptable.

7.4 Discussion

The system demonstrates that a practical, secure, and usable electronic voting platform can be built using commodity hardware (USB fingerprint scanner) and open-source software within a student project timeframe. The key design decision — to enforce all security constraints server-side — proved to be the right approach: during testing, attempts to bypass the schedule check and the double-vote check by sending raw API requests were all correctly rejected.

The fingerprint fallback mechanism was essential for development, allowing the team to test all voter flows without physical access to the scanner. The fallback is transparently logged and designed to be easily disabled in production by checking is_bwapi_alive () at startup.

The admin panel evolved significantly during development based on practical needs. The initial design had no voter search or CSV export; these were added after testing revealed that administrators needed to identify specific voters quickly and archive records externally. This reflects an iterative development approach that responded to emerging requirements.

7.5 Limitations Encountered

- Single election constraint: The system supports only one active election at a time. The voting_schedule collection holds a single document (upserted on each set schedule call). Supporting concurrent elections would require a multi-election data model with election_id foreign keys.
- Non-atomic vote recording: As discussed in Section 5.4.1, the vote counter increment and has Voted update are not wrapped in a MongoDB transaction (requires replica set). For low-concurrency institutional elections this is acceptable, but it should be addressed before deployment in high-concurrency scenarios.
- Hardware dependency: The full fingerprint feature requires Windows and SecuGen FPWebHost. Non-Windows deployments and deployments without the scanner are limited to development/fallback mode, which is not suitable for production elections.
- No voter anonymity: Votes are linked to voters via the candidate_id is implicitly tied to the voter through the voted at timestamp. True ballot anonymity requires cryptographic techniques (e.g., blind signatures) not implemented here.
- No audit log: There is no immutable log of all system events. An audit table recording every login attempt, vote submission, and admin action would improve accountability.

Chapter 8: Conclusion & Future Work

8.1 Conclusion

This project successfully designed and implemented a fingerprint-based electronic voting system that addresses the core weaknesses of paper-based and password-only digital voting: impersonation, double voting, and lack of real-time transparency. The system integrates the SecuGen HU20 biometric scanner through a Flask proxy layer, enforces a server-side time-locked voting window, and provides a live results dashboard that requires no authentication for public viewing.

All eight project objectives were fully achieved. Twenty functional test cases passed with no critical failures. The system is deployable on any Windows machine with Python 3.10+, MongoDB 7.x, and the SecuGen FPWebHost service running on port 8443. A graceful fallback mode ensures that all voter and admin flows can be demonstrated and tested without physical scanner hardware.

The project demonstrates that modern open-source tooling (Flask, MongoDB, Chart.js, Vanilla JS) is sufficient to deliver a production-quality secure voting system without resorting to heavy enterprise frameworks, making it an accessible template for institutions seeking to modernize their electoral processes.

8.2 Limitations

The main limitations of the current system are: (1) single-election-at-a-time data model, (2) non-atomic vote recording without a MongoDB replica set, (3) Windows-only fingerprint hardware dependency, (4) lack of true voter anonymity, and (5) absence of an immutable audit log. These are documented in Section 7.5 with known remediation paths.

8.3 Future Enhancements

The following enhancements are proposed for future versions of the system:

1. Multi-election support: Refactor the data model to include an election_id field on all relevant collections, allowing multiple concurrent elections to be managed from a single admin panel.
2. MongoDB replica set deployment: Configure MongoDB as a three-node replica set to enable multi-document transactions, providing true atomic vote recording.
3. Mobile fingerprint integration: Explore Webathon / FIDO2 for mobile devices, allowing fingerprint authentication via the device's built-in biometric sensor without a USB scanner.
4. End-to-end verifiable receipts: Implement a cryptographic receipt system (similar to Helios) where each voter receives a token they can use to verify their vote was counted without revealing their choice.
5. SMS / email confirmation: Send a vote confirmation SMS or email to the voter after a successful ballot is cast, providing an additional audit mechanism.

6. Audit log: Add an immutable audit collection recording every login attempt, vote cast, admin action, and schedule change with timestamps and IP addresses.
7. Progressive Web App (PWA): Add a service worker and manifest to allow the voter interface to be installed as a home-screen app on mobile devices.
8. Role-based admin access: Extend the admin model to support multiple admin accounts with different permission levels (e.g., read-only observer vs. full administrator).

8.4 Social Impact and Applications

Beyond its immediate institutional use case, the E-Voting System demonstrates a pattern that could be adapted for a range of social applications: student government elections at universities, professional society board elections, municipal-level polling in areas with limited physical polling infrastructure, and NGO member voting. By lowering the cost and technical barrier to biometric-authenticated voting, the project contributes to a broader ecosystem of trustworthy digital democracy tools.

The use of the OpenStreetMap Nominatim API for location data also demonstrates that it is possible to build location-aware civic technology without relying on proprietary paid APIs, which is particularly important for community-funded and open-source civic projects.

References

[1] SecuGen Corporation. (2024). SecuGen Web API (BWAPI) Developer Guide. Retrieved from https://secugen.com/support/

[2] Flask Development Team. (2024). Flask Documentation, Version 3.0. Retrieved from https://flask.palletsprojects.com/

[3] MongoDB Inc. (2024). MongoDB Manual — Version 7.0. Retrieved from https://www.mongodb.com/docs/manual/

[4] ISO/IEC 19794-2:2011. (2011). Information Technology — Biometric Data Interchange Formats — Part 2: Finger Minutiae Data. International Organization for Standardization.

[5] Nyst, C., & Monaco, N. (2018). State-Sponsored Disinformation in the 2018 Elections and the Future of E-Voting. Freedom House.

[6] Springall, D., Finke Nauer, T., Durumeric, Z., Kit cat, J., Hurst, H., MacAlpine, M., & Halderman, J. A. (2014). Security Analysis of the Estonian Internet Voting System. Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security.

[7] Bernhard, M., Benaloh, J., Halderman, J. A., Rivest, R. L., Ryan, P. Y. A., Stark, P. B., Vora, P. L., Wallach, D. S., & Wack, M. (2017). Public Evidence from Secret Ballots. E-Vote-ID 2017.

[8] Chart.js Contributors. (2024). Chart.js Documentation, Version 4.4. Retrieved from https://www.chartjs.org/docs/

[9] OpenStreetMap Foundation. (2024). Nominatim API Documentation. Retrieved from https://nominatim.org/release-docs/develop/api/

[10] Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference.

Appendix

Appendix A – Installation and Setup Guide

A.1 Prerequisites

- Python 3.10 or higher (https://www.python.org/downloads/)
- MongoDB 7.x Community Edition (https://www.mongodb.com/try/download/community)
- SecuGen FPWebHost installed and running on port 8443 (Windows only; obtain from SecuGen support)
- Modern web browser (Chrome 100+ or Firefox 100+)

A.2 Quick Start (Windows)

1. Clone or extract the project to a local folder.
2. Open a Command Prompt in the project root.
3. Double-click start.bat — this will: create a Python virtual environment, install dependencies, check MongoDB, start the Flask server, and open the browser at http://localhost:5000.
4. Alternatively, run manually: cd Backend, python -m venv, venv\Scripts\activate, pip install -r requirements.txt, python app.py.
5. Navigate to http://localhost:5000/api/ping to verify the backend is running.
6. Default admin credentials: username admin, password admin123. Change these in Backend/.env before use.

A.3 Environment Variables

Variable	Default Value	Description
SECRET_KEY	Sarfraz	Flask session secret key — change in production
MONGO_URI	mongodb://localhost:27017/	MongoDB connection URI
ADMIN_USERNAME	admin	Default admin username
ADMIN_PASSWORD	admin123	Default admin password — change immediately

Variable	Default Value	Description
BWAPI_URL	https://localhost:8443	SecuGen BWAPI service URL
BWAPI_ORIGIN	http://localhost:5000	Origin header for BWAPI requests
FP_MATCH_THRESHOLD	40	Minimum score (0-199) to accept as fingerprint match

Appendix B – API Endpoint Reference

A complete API reference is provided in Table 4.5 (Chapter 4). All endpoints return JSON. Authentication is handled via HTTP-only session cookies set by Flask. For endpoints requiring a voter session, the client must include credentials: 'include' in fetch () calls. For admin endpoints, a separate admin session must be established via POST /api/admin/login.

Error responses follow the format: {“error": "Human-readable message”}. Success responses include {“success": true} plus relevant data fields. HTTP status codes follow REST conventions: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Processable Entity, 503 Service Unavailable.

Appendix C – Storage Schema Reference

Complete MongoDB collection schemas are documented in Tables 4.1–4.4 in Chapter 4. Two MongoDB indexes are created automatically on application startup: a unique index on voters. Cnic and a unique index on voters. Email. These indexes ensure O (log n) lookup performance for login queries and prevent duplicate registrations at the database level, independent of application-layer checks.

The voting_schedule collection always contains at most one document (enforced by the replace one with upset=True pattern). The candidate’s collection has no upper limit; the admin interface supports deletion at any time outside the active voting window.